

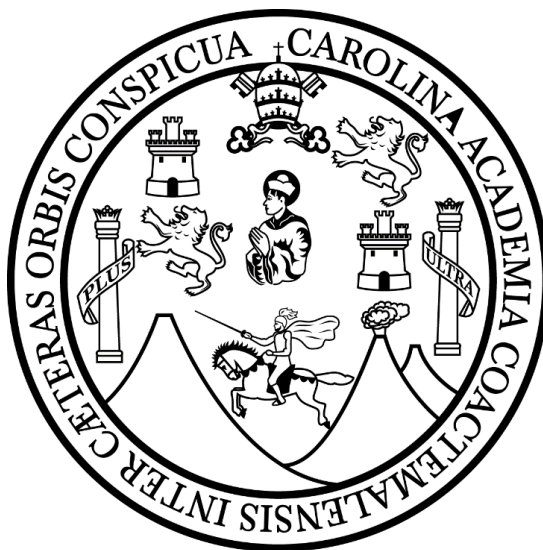
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERÍA

ESCUELA DE CIENCIAS Y SISTEMAS

INTELIGENCIA ARTIFICIAL 1

SECCIÓN A



Manual Tecnico

Proyecto Fase 1 Grupo 9	
201504464	Herberth Abisai Avila Ruiz
202000562	Pedro Luis Pu Tavico
201602983	Jaime Alejandro Armira Us

Tabla de Contenido

Introducción.....	3
Tecnologías y Herramientas Utilizadas.....	3
Estructura del Proyecto.....	3
Implementación del Modelo.....	3
Cómo Ejecutar el Proyecto.....	3
Documentación de Código.....	4
Mantenimiento y Mejora.....	4

Introducción

Este chatbot ha sido desarrollado para interactuar en tiempo real en español e inglés, permitiendo conversaciones sobre diversos temas como arte, deportes, tecnología, comida, y más. Está diseñado para ser accesible desde cualquier navegador web moderno y funciona de manera local, sin requerir una conexión a internet para su uso básico.

Tecnologías y Herramientas Utilizadas

- **Lenguaje de Programación:** JavaScript (Ejecutado en el navegador) y Python (para el entrenamiento y manejo del modelo).
- **Bibliotecas:**
 - **TensorFlow.js:** Para la inferencia del modelo en el navegador.
 - **Keras:** Para el entrenamiento de la red neuronal.
 - **NLKT:** Para procesamiento del lenguaje natural
 - **HTML/CSS** para la interfaz de usuario.
 - **JSON:** Para manejar intenciones y patrones del chatbot.
- **Desarrollo Web:** Utilización de HTML5, CSS3 y JavaScript para la construcción de la interfaz gráfica y la lógica de interacción.

Estructura del Proyecto

La estructura del proyecto está organizada en varias carpetas y archivos que manejan diferentes aspectos del chatbot:

- **Fase2/index.html:** Archivo HTML que define la estructura de la interfaz de usuario, donde el usuario puede interactuar con el chatbot.
- **Fase2/style.css:** Archivo para la personalización de la apariencia de la interfaz.
- **Fase2/script.js:** Maneja la lógica del chatbot, incluida la carga del modelo y la interacción del usuario.
- **Fase2/modelo/chatbot.js:** Código Python para preprocesar entradas y entrenar el modelo de red neuronal.
- **Fase2/modelo/training.py:** Código Python para el entrenamiento del modelo, generación de “bag of words” y clases.
- **Fase2/modelo/intents.json:** Archivos JSON (intents_es.json, intents_en.json) que almacena los intents (intenciones) y patrones de entrada, junto con las respuestas posibles.

Cada intent tiene un "tag" que representa la categoría de la consulta del usuario, un conjunto de "patterns" que son frases típicas que corresponden a ese intent, y las "responses" posibles que el chatbot puede dar. Todo esto para español e inglés.

- **Fase2/modelo/words.json, classes.json, chatbot_model.h5:** Archivos que contienen el modelo entrenado y las clases correspondientes.

Implementación del Modelo

El modelo se implementa en **Keras** y se exporta a un formato compatible con **TensorFlow.js** para su uso en el navegador. A continuación se describen las partes clave:

1. Entrenamiento del Modelo:

- El archivo `training.py` combina las intenciones en español e inglés para entrenar una red neuronal. Las frases de entrada se tokenizan, lematizan y se transforman en una representación de "bag of words".
- La red neuronal se entrena para clasificar las intenciones del usuario.
- El modelo se guarda como `chatbot_model.h5` y luego se convierte a **TensorFlow.js** para ser utilizado en el navegador.

2. Interacción con el Navegador:

- En `script.js`, el archivo `model.json` se carga utilizando TensorFlow.js. Las entradas del usuario se pre procesan con funciones que eliminan tildes y expanden contracciones.
- El modelo predice la intención y devuelve una respuesta del archivo de intenciones correspondiente (`intents_es.json` o `intents_en.json`).

Cómo Ejecutar el Proyecto

1. Clona el repositorio desde GitHub.
2. Abre el archivo **index.html** en cualquier navegador web.
3. El modelo estará listo para interactuar con las entradas del usuario.

Documentación de Código

Archivo `index.html`:

- Estructura básica del chatbot, incluye el campo de entrada de texto y los contenedores para mostrar los mensajes.

```
<!DOCTYPE html>

<html lang="es">

  <head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width,
initial-scale=1.0">

    <link rel="stylesheet" href="./style.css">

    <title>Chatbot</title>

  </head>

  <body>

    <div class="chat-container">

      <div class="chat-header">

        Chatbot

      </div>

      <div class="chat-messages"
id="chat-messages"></div>

      <div class="chat-input">

        <input type="text" id="chat-input"
placeholder="Escribe un mensaje..." autocomplete="off">

        <button id="send-button">Enviar</button>

      </div>

    </div>

  </body>

</html>
```

```

        </div>

    </div>

    <script
src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs"></scri
pt>

    <script src="./script.js"></script>

</body>

</html>

```

Archivo **script.js**:

- **loadResources()**: Carga el modelo y los archivos JSON necesarios.

```

async function loadResources() {

    model = await
tf.loadLayersModel('./modelo/model_js/model.json');

    const wordsResponse = await
fetch('./modelo/words.json');

    words = await wordsResponse.json();

    const classesResponse = await
fetch('./modelo/classes.json');

    classes = await classesResponse.json();

    const intentsResponse = await
fetch('./modelo/intents_combine.json');

    intents = await intentsResponse.json();
}

```

```

    const contractionsResponse = await
fetch('./modelo/contractions.json');

    contractions = await contractionsResponse.json();
}

```

- **bagOfWords(sentence):** Convierte una oración en una representación numérica basada en las palabras.

```

function bagOfWords(sentence) {

    let texto = quitarTildes(sentence);

    texto = expandirContracciones(texto);

    const sentenceWords = texto.toLowerCase().split(' ');

    const bag = new Array(words.length).fill(0);

    sentenceWords.forEach(word => {

        const index = words.indexOf(word);

        if (index !== -1) {

            bag[index] = 1;

        }

    });

    return tf.tensor2d([bag]);
}

```

- **predictClass(sentence):** Usa el modelo para predecir la intención de la oración.

```

async function predictClass(sentence) {

    const bow = bagOfWords(sentence);

    const predictions = await model.predict(bow).data();
}

```

```

    const maxIndex =
predictions.indexOf(Math.max(...predictions));

    return classes[maxIndex];
}

```

- **getResponse(tag):** Selecciona una respuesta aleatoria del intent correspondiente.

```

function getResponse(tag) {

    const intent = intents.intents.find(intent =>
intent.tag === tag);

    if (intent) {

        const responses = intent.responses;

        return responses[Math.floor(Math.random() *
responses.length)];

    }

    return "No entiendo tu mensaje, intenta de nuevo.";
}

```

- **enviarMensaje():** Maneja la interacción del usuario con el chatbot, mostrando el mensaje del usuario y la respuesta generada.

```

async function enviarMensaje() {

    const input = document.getElementById('chat-input');

    const mensajeUsuario = input.value.trim();

    if (!mensajeUsuario) return;

    mostrarMensaje(mensajeUsuario, 'user');

    const respuestaBot = await respuesta(mensajeUsuario);

```



```

mostrarMensaje(respuestaBot, 'bot');

input.value = '';

}

```

Archivo **chatbot.py**:

- **bag_of_words(sentence)**: Convierte las palabras de una oración en una representación de "bag of words".

```

#Convertimos la información a unos y ceros según si están
#presentes en los patrones

def bag_of_words(sentence):

    sentence_words = clean_up_sentence(sentence)

    bag = [0]*len(words)

    for w in sentence_words:

        for i, word in enumerate(words):

            if word == w:

                bag[i]=1

    print(bag)

    return np.array(bag)

```

- **predict_class(sentence)**: Predice la intención de una oración basada en el modelo entrenado.

```

#Predecimos la categoría a la que pertenece la oración

def predict_class(sentence):

    bow = bag_of_words(sentence)

    res = model.predict(np.array([bow]))[0]

    max_index = np.where(res == np.max(res))[0][0]

    category = classes[max_index]

```

```
return category
```

- **get_response(tag, intents_json):** Selecciona una respuesta basada en la intención predicha.

```
#Obtenemos una respuesta aleatoria

def get_response(tag, intents_json):

    list_of_intents = intents_json['intents']

    result = ""

    for i in list_of_intents:

        if i["tag"]==tag:

            result = random.choice(i['responses'])

            break

    return result
```

Archivo **training.py**:

- **expand_contractions(text, contractions_dict):** Expande las contracciones en una oración.

```
def expand_contractions(text, contractions_dict):

    pattern = re.compile(r'\b(' + '|'.join(re.escape(key)
for key in contractions_dict.keys()) + r')\b')

    return pattern.sub(lambda match:
contractions_dict[match.group()], text)
```

- **Red Neuronal:** Implementa una red neuronal densa para la clasificación de intenciones.

```
#Creamos la red neuronal

model = Sequential()

model.add(Dense(128, input_shape=(len(train_x[0]),),
name="input_layer", activation='relu'))
```

```

model.add(BatchNormalization())

model.add(Dropout(0.5, name="input_layer1"))

model.add(Dense(64, name="input_layer2",
activation='relu'))

model.add(BatchNormalization())

model.add(Dropout(0.5, name="input_layer3"))

model.add(Dense(len(train_y[0]), name="output_layer",
activation='softmax'))

```

- **Entrenamiento:** Ajusta el modelo utilizando las frases y patrones de entrada.

```

#Entrenamos el modelo y lo guardamos

model.fit(np.array(train_x), np.array(train_y),
epochs=500, batch_size=8, validation_split=0.2,
callbacks=[early_stopping, reduce_lr])

model.save("chatbot_model.h5")

model.summary()

tfjs.converters.save_keras_model(model, 'model_js')

```

Mantenimiento y Mejora

- **Entrenamiento adicional:** Para mejorar el modelo, se puede añadir más datos al conjunto de entrenamiento y ajustar los parámetros de la red neuronal.
- **Mejora del rendimiento:** Optimizar el tamaño del modelo y ajustar los hiperparámetros para mejorar su desempeño en dispositivos con recursos limitados.

Entrenamiento del Modelo

El modelo de chatbot fue entrenado utilizando **Python** y la biblioteca **Keras** para crear una red neuronal que clasifica las intenciones del usuario en función de las entradas proporcionadas. El proceso de entrenamiento incluye la preprocesamiento de texto y la conversión de las oraciones en una representación numérica que la red puede entender.

Pasos para el Entrenamiento del Modelo:

1. Carga de Intenciones:

- Se combinan las intenciones definidas en los archivos `intents_es.json` y `intents_en.json` mediante la función `load_intents` del archivo `training.py`. Estos archivos contienen los patrones de entrada (frases comunes) y las posibles respuestas que el chatbot puede proporcionar.

```
def load_intents(files, dir_path):  
  
    combine_intents={"intents":[]}  
  
    for file in files:  
  
        with open(dir_path+os.sep+file, 'rb') as f:  
  
            intents = json.load(f)  
  
    combine_intents["intents"].extend(intents["intents"])  
  
    return combine_intents  
  
dir_path='./file_intents'  
  
files_list = os.listdir(dir_path+os.sep)  
  
intents = load_intents(files_list, dir_path)
```

2. Preprocesamiento del Texto:

- Se utilizan las bibliotecas **NLTK** y **WordNetLemmatizer** para tokenizar las oraciones, lo que significa dividir cada oración en palabras individuales.
- El texto se lematiza, es decir, se reduce cada palabra a su forma raíz. Por ejemplo, las palabras "corriendo", "corre" y "correr" se convierten en "correr" para uniformizar el análisis.
- También se expanden las contracciones utilizando el archivo **contractions.json**, donde se definen contracciones comunes (por ejemplo, "I'm" se expande a "I am").

```
words = [lemmatizer.lemmatize(word) for word in words if
word not in ignore_letters]

words = sorted(set(words))

pickle.dump(words, open('words.pkl', 'wb'))

pickle.dump(classes, open('classes.pkl', 'wb'))
```

3. Representación de "Bag of Words":

- Cada oración es convertida a una representación de "bag of words" en la que se crea un vector de unos y ceros indicando la presencia o ausencia de palabras específicas en la oración. Esta representación simplifica las oraciones para que puedan ser procesadas por la red neuronal.

4. Definición de la Red Neuronal:

- El modelo se define utilizando la API **Keras**. La red tiene las siguientes características:
 - Una capa de entrada con 128 neuronas y activación **ReLU**.
 - Una capa oculta con 64 neuronas y activación **ReLU**.
 - Una capa de salida con tantas neuronas como clases de intenciones, usando la función de activación **Softmax** para obtener probabilidades.
 - Se utilizan capas de **Dropout** y **Batch Normalization** para evitar el sobreajuste y mejorar la estabilidad del entrenamiento.

5. Compilación y Optimización:

- El modelo utiliza el optimizador **SGD** (Gradiente Descendente Estocástico) con un decaimiento exponencial de la tasa de aprendizaje para ajustar el modelo de manera eficiente. La función

de pérdida utilizada es **Categorical Crossentropy**, ya que se trata de un problema de clasificación.

- Se incluye una estrategia de **Early Stopping** para detener el entrenamiento cuando el modelo deja de mejorar, evitando así el sobreajuste.

```
#Creamos el optimizador y lo compilamos

sgd = SGD(learning_rate=lr_schedule , momentum=0.9,
nesterov=True)

model.compile(loss='categorical_crossentropy', optimizer =
sgd, metrics = ['acc'])

# Definimos el callback de early stopping para evitar el
sobreajuste

early_stopping = EarlyStopping(monitor='val_loss',
patience=150,
restore_best_weights=True,min_delta=0.001,verbose=1)

#reduce lr on plateau

reduce_lr = ReduceLROnPlateau(monitor='val_loss',
factor=0.2, patience=50, min_lr=0.0001)
```

6. Entrenamiento y Validación:

- El modelo se entrena utilizando las frases preprocesadas y sus respectivas etiquetas (intenciones). Se utilizan 500 épocas y un tamaño de lote de 8 ejemplos por iteración. Durante el entrenamiento, se utiliza un 20% de los datos como conjunto de validación para monitorear el rendimiento del modelo en datos no vistos.

```
#Entrenamos el modelo y lo guardamos

model.fit(np.array(train_x), np.array(train_y),
epochs=500, batch_size=8,validation_split=0.2,
callbacks=[early_stopping,reduce_lr])
```

```

model.save("chatbot_model.h5")

model.summary()

tfjs.converters.save_keras_model(model, 'model_js')

```

7. Exportación del Modelo:

- Una vez finalizado el entrenamiento, el modelo se guarda en formato **H5** (`chatbot_model.h5`) para su reutilización. Además, se convierte a **TensorFlow.js** mediante la función `tfjs.converters.save_keras_model(model, 'model_js')`, lo que permite su uso en el navegador.

```

#Entrenamos el modelo y lo guardamos

model.fit(np.array(train_x), np.array(train_y),
epochs=500, batch_size=8, validation_split=0.2,
callbacks=[early_stopping, reduce_lr])

model.save("chatbot_model.h5")

model.summary()

tfjs.converters.save_keras_model(model, 'model_js')

```

8. Archivos Generados:

- `words.pkl`: Contiene las palabras únicas usadas en el entrenamiento.
- `classes.pkl`: Contiene las clases (intenciones) identificadas.
- `chatbot_model.h5`: El modelo entrenado que se utiliza para la inferencia en tiempo real.
- `model.json`: La versión de TensorFlow.js del modelo para su uso en el navegador.

Alojar el Chatbot en GitHub Pages

GitHub Pages es una excelente opción para alojar sitios web estáticos, y se puede utilizar para alojar el chatbot directamente desde un repositorio de GitHub. Sigue los siguientes pasos para desplegar el proyecto en GitHub Pages.

Pasos para Alojar el Chatbot en GitHub Pages

1. Crea un Repositorio en GitHub:

- Inicia sesión en tu cuenta de GitHub.
- Haz clic en el botón **New** para crear un nuevo repositorio.
- Asigna un nombre al repositorio (por ejemplo, "chatbot").
- Marca la opción **Public** para que el repositorio sea accesible a todos.
- Opcional: Agrega una descripción al repositorio.
- Haz clic en **Create repository**.

2. Sube los Archivos del Proyecto:

- En el nuevo repositorio, haz clic en el botón **Add file** y selecciona **Upload files**.
- Sube los siguientes archivos de tu proyecto:
 - `index.html`
 - `style.css`
 - `chatbot.js`
 - Los archivos de la carpeta `modelo/` (incluidos `model.onnx`, `intents.json`, etc.).
- Luego haz clic en **Commit changes** para guardar los archivos en el repositorio.

3. Configura GitHub Pages:

- Una vez que los archivos están subidos, ve a la pestaña **Settings** del repositorio.
- Desplázate hacia abajo hasta la sección **Pages**.
- En **Source**, selecciona la rama `main` (o `master` dependiendo del nombre de la rama principal de tu repositorio).
- En el menú desplegable que aparece al lado, selecciona la opción `/ (root)` para que el contenido del repositorio raíz se publique.
- Haz clic en **Save**.

4. Verifica el Despliegue:

- GitHub generará una URL donde se alojará tu chatbot (normalmente en el formato `https://<tu-usuario>.github.io/<nombre-del-repositorio>/`).
- Espera unos minutos mientras GitHub Pages despliega tu sitio.

- Una vez completado, abre el enlace para verificar que el chatbot está funcionando correctamente.

Ejemplo de URL Final:

Una vez que el despliegue esté completo, deberías poder acceder al chatbot desde una URL como la siguiente (dependiendo de tu nombre de usuario y el nombre del repositorio):

- <https://<tu-usuario>.github.io/chatbot/>